Project 1 Background: What Are You Training?

If you have taken a machine learning class, skip this file. If not, read it before you start on `p1.go`. Nothing here is graded. Everything past this document assumes you know these words.

The model

A **model** is a rule that takes input numbers and returns an output number. The rule in this project is:

```
prediction = w[0]*x[0] + w[1]*x[1] + ... + w[15]*x[15]
```

You have 16 input numbers (called **features**, `x`) and 16 **weights** (`w`). You multiply each feature by its weight, add them all up, and that sum is the model's prediction.

The features come in as `example.X` in the code. The weights are what you are trying to learn. If you pick good weights, the model predicts well. If you pick bad weights, it does not. In this project `w` is a `[]float64` of length `D = 16`. That is the whole model.

The number 16 does not matter. Real models have millions or billions of weights. The same steps work at any size.

The dataset

We do not use real data. We generate fake data from a seed, so the same seed always gives the same data on any machine.

The data generator in `sealed.go` (function `Load`) works like this:

1. Pick a hidden "true" weight vector `w_true` from the seed. You do not see `w_true` in your code. If you did, the project would be trivial. 2. Draw feature vectors `X` at random. 3. Compute `Y = w_true · X + small random noise`. `Y` is what the model should predict from `X`.

So the training data is a list of `(X, Y)` pairs. `Y` is almost a straight-line function of `X`, plus a little noise. Your job is to find a `w` that predicts `Y` from `X` well, using only the `(X, Y)` pairs. You never see `w_true`, but if you train correctly your `w` will be close to it.

The dataset has 8 shards, 1024 examples per shard. That is 8,192 examples in total. `NewBatcher` shuffles them once (using the seed, so the order is the same every run) and gives you a fresh batch of `BatchSize` examples every time you call `Next(step)`.

Loss

We need a single number that says how bad the current `w` is. The standard choice is **mean squared error**: for each example, subtract the prediction from the true `Y`, square the result, then average over all examples in the batch:

```
loss(w, batch) = average of (w · X - Y)^2 over the batch
```

If `w` equals `w_true`, the loss is very small (not zero, because we added noise). If `w` is garbage, the loss is large.

Training is the process of making the loss smaller.

Gradient

The **gradient** of the loss is a vector of the same length as `w`. It answers a question for each weight: "if I nudge this weight up by a tiny amount, does the loss go up or down, and by how much?"

If a gradient entry is positive, nudging that weight up makes the loss go up. If negative, nudging it up makes the loss go down.

To make the loss smaller, we move `w` in the direction *opposite* the gradient. This is called **gradient descent**: read the slope, take a small step downhill, repeat. After a few hundred steps, `w` is close to `w_true`.

You do not write the gradient code. The sealed function `Grad(w, batch)` computes it for you. It returns the *sum* of the per-example gradients over the batch, not the mean. You divide by the batch size when you apply the update.

One step, with real numbers

Say you have one example: `X = [1, 2, 0, 0, ..., 0]` (only the first two features are nonzero), true `Y = 3`. Start with `w = [0, 0, ..., 0]`.

- Prediction: `w · X = 0`.
- Error: `prediction - Y = -3`.
- Per-example gradient of `(w·X - Y)^2` is `2 · (w·X - Y) · X`. That works out to `2 · (-3) · X = [-6, -12, 0, ..., 0]`.
- Update rule: `w[i] -= LR · grad[i] / batchSize`. With `LR = 0.01` and batch size 1, `w` becomes `[0.06, 0.12, 0, ..., 0]`.

Check: new prediction on the same example is `0.06 · 1 + 0.12 · 2 = 0.30`. Still not 3, but closer than 0. Repeat this a few thousand times on real batches and `w` will end up predicting `Y` well.

Minibatches

You do not have to use one example at a time or all 8,192 at once. Using all of them gives you the most accurate gradient but is slow. Using one at a time is fast but very noisy.

Minibatch SGD picks a middle ground. Each step uses a small random batch of a few dozen to a few hundred examples. Cheap per step, accurate enough per step. The noise from small batches also helps the model find a better final `w`.

In this project the batch size is a config field (`cfg.BatchSize`). The graded configs use batches around 64.

What Project 1 does

Once you understand the loop above (draw a batch, compute a gradient, apply the update, repeat), everything the project asks you to do is about running that loop in parallel.

- **Part 1** runs the loop in one goroutine on one CPU core. If you understand the "one step" section, you can write Part 1.
- **Part 2** splits each batch across `N` goroutines. Each goroutine computes the gradient for its slice of the batch. The goroutines add their gradients together. Then each goroutine applies the same update to its own copy of `w`. This is called **data-parallel training**.
- **Part 3** puts each goroutine in a separate process, talking over RPC. The graders kill some of these processes during training. Your code must still return the right weights.

Once you have written this, you have written the core of every real distributed-training system. What TensorFlow, PyTorch, and JAX do is this same loop at a much larger scale, with more engineering around it.

Words used in the code

- **weight, w** -- the vector of numbers the model is learning.
- **feature, X** -- the input to the model.
- **label, Y** -- the number the model should predict.
- **loss** -- one number that says how bad `w` is. Lower is better.
- **gradient** -- vector, same shape as `w`, points in the direction of higher loss.
- **batch, minibatch** -- a small group of examples used for one step.
- **step** -- one iteration: batch, gradient, apply.
- **epoch** -- one full pass through the whole dataset. This project does not care about epochs; it just runs a fixed number of steps.
- **LR (learning rate)** -- how far to move `w` on each step.
- **converge** -- reach a `w` where the loss stops going down. Training is done.
- **initial weights** -- what `w` looks like before training. In this project, small random numbers from `Init(seed)`.

If you are still lost

Come to office hours, and please attend recitations to ask project-related questions.